

Submission Report

1. Planning problem specification

Initial state: {At(Robot,A), At(Package,A)}. Goal: At(Package,C). Actions include the four connected moves, pickup actions, and Drop(Package,C), with explicit preconditions/effects.

2. Manually constructed plan

```
S0: At(Robot,A), At(Package,A)
1. Pickup(Package,A)
S1: At(Robot,A), Holding(Package)
2. Move(A,B)
S2: At(Robot,B), Holding(Package)
3. Move(B,C)
S3: At(Robot,C), Holding(Package)
4. Drop(Package,C)
S4: At(Robot,C), At(Package,C)
```

3. Prompt used

I want to implement a simple planning agent in Python. Represent a state as a set of logical propositions. Each action should contain a name, positive/negative preconditions, and positive/negative effects. An action is applicable only when its preconditions are satisfied. Use BFS to find a sequence achieving a specified goal, detect failure, print the plan and states, explain assumptions, and run the warehouse problem.

4. Test results

```
TEST A: Solvable
Plan: Pickup(Package,A) -> Move(A,B) -> Move(B,C) -> Drop(Package,C)
S0: ['At(Package,A)', 'At(Robot,A)']
S1: ['At(Robot,A)', 'Holding(Package)']
S2: ['At(Robot,B)', 'Holding(Package)']
S3: ['At(Robot,C)', 'Holding(Package)']
S4: ['At(Package,C)', 'At(Robot,C)']
TEST B: Impossible: No plan found
TEST C: Irrelevant actions: No plan found
```

5. Reflection

The LLM helps translate a precise logical specification into code, but preconditions, state transitions, goal satisfaction, and failure cases must be independently verified. Logical reasoning determines which actions are applicable; BFS searches among applicable action sequences.